

Triggers and Procedures

STORED PROCEDURES

PROCEDURE TO DISPLAY ALL CUSTOMERS

The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane displays the database '2520080053_ruhitha'. The main editor shows a SQL query to create a stored procedure named 'GetAllCustomers()' which selects all records from the 'Customer' table. The procedure is then called. Below the editor, the 'Result Grid' shows the output of the procedure call, displaying a list of customers with columns: Customer_ID, Customer_Name, Phone, Email, and City. The 'Output' pane shows the execution log, indicating that the procedure was successfully created and executed, returning 10 rows.

Customer_ID	Customer_Name	Phone	Email	City
101	Ravi Kumar	9876543210	ravi@gmail.com	Hyderabad
102	Priya Sharma	9876543211	priya@gmail.com	Vijayawada
103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
104	Sneha Rao	9876543213	sneha@gmail.com	Chennai
105	Kiran Kumar	9876543214	kiran@gmail.com	Hyderabad
106	Anil Kumar	9876543215	anil@gmail.com	Delhi
107	Meena Reddy	9876543216	meena@gmail.com	Mumbai
108	Rahul Sharma	9876543217	rahul@gmail.com	Pune
109	Lakshmi Devi	9876543218	lakshmi@gmail.com	Hyderabad
110	Suresh Babu	9876543219	suresh@gmail.com	Vijayawada

PROCEDURE TO DISPLAY ACCOUNT DETAILS FOR A GIVEN ACCOUNT NUMBER

The screenshot shows the MySQL Workbench interface. The main editor displays a SQL query to create a stored procedure named 'GetAccountDetails()' which takes an account number as input and selects all records from the 'Account' table where the account number matches the input. The procedure is then called. Below the editor, the 'Output' pane shows the execution log, indicating that the procedure was successfully created and executed, returning 0 rows.

#	Time	Action	Message
7	10:04:47	CALL GetAllCustomers()	10 row(s) returned
8	10:08:22	CREATE PROCEDURE GetAccountDetails(IN p_Account_No INT) BEGIN SELECT * FROM Account...	Error Code: 1064. You
9	10:08:58	CREATE PROCEDURE GetAccountDetails(IN p_Account_No INT) BEGIN SELECT * FROM Account...	0 row(s) affected

2520080053_Ruhitha (252008005.x) 2520080053_Ruhitha (25200... x) Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator: Filter objects

2520080053_ruhitha

- Tables
- Views
- Stored Procedures
- Functions
- bookflow_db
- kludb
- sys

Administration Schemas

Information

No object selected

Object Info Session

Query 1 SQL File 1* SQL File 2* SQL File 3* x

```

24 CREATE PROCEDURE GetAccountDetails(
25     IN p_Account_No INT
26 )
27 BEGIN
28     SELECT *
29     FROM Account
30     WHERE Account_No = p_Account_No;
31 END //
32 DELIMITER ;
33 CALL GetAccountDetails(10001);

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	20000.00	Hyderabad

Result 6 x

Output

Action Output

#	Time	Action	Message
8	10:08:22	CREATE PROCEDURE GetAccountDetails(IN p_Account_No INT) BEGIN SELECT * FROM Account...	Error Code: 1064.
9	10:08:58	CREATE PROCEDURE GetAccountDetails(IN p_Account_No INT) BEGIN SELECT * FROM Account...	0 row(s) affected
10	10:09:22	CALL GetAccountDetails(10001)	1 row(s) returned

PROCEDURE TO FIND CUSTOMER ACCOUNTS

2520080053_Ruhitha (252008005.x) 2520080053_Ruhitha (25200... x) Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator: Filter objects

2520080053_ruhitha

- Tables
- Views
- Stored Procedures
- Functions
- bookflow_db
- kludb
- sys

Administration Schemas

Information

No object selected

Object Info Session

Query 1 SQL File 1* SQL File 2* SQL File 3* x

```

40 SELECT
41     C.Customer_ID,
42     C.Customer_Name,
43     A.Account_No,
44     A.Account_Type,
45     A.Balance,
46     A.Branch
47 FROM Customer C
48 JOIN Account A
49 ON C.Customer_ID = A.Customer_ID
50 WHERE C.Customer_ID = p_Customer_ID;
51 END //
52 DELIMITER ;
53 CALL GetCustomerAccounts(101);

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Customer_ID	Customer_Name	Account_No	Account_Type	Balance	Branch
101	Ravi Kumar	10001	Savings	20000.00	Hyderabad

Result 7 x

Output

Action Output

#	Time	Action	Message
11	10:11:33	CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT) BEGIN SELECT C.Cust...	0 row(s) affected
12	10:11:33	CALL GetCustomerAccounts(101)	1 row(s) returned

PROCEDURE TO DEPOSIT MONEY

The screenshot shows the MySQL Workbench interface with the '2520080053_Ruhitha' database selected. The 'SCHEMAS' panel on the left shows the database structure. The main editor displays the following SQL code:

```
46 A.Branch
47 FROM Customer C
48 JOIN Account A
49 ON C.Customer_ID = A.Customer_ID
50 WHERE C.Customer_ID = p_Customer_ID;
51 END //
52 DELIMITER ;
53 CALL GetCustomerAccounts(101);
54
55
56 DELIMITER //
57 CREATE PROCEDURE DepositMoney(
58 IN p_Account_No INT,
59 IN p_Amount DECIMAL(12,2)
60 )
61 BEGIN
62
63 UPDATE Account
64 SET Balance = Balance + p_Amount
65 WHERE Account_No = p_Account_No;
66
67 END //
68 DELIMITER ;
69 CALL DepositMoney(10001, 5000);
```

The 'Output' panel at the bottom shows the execution results:

#	Time	Action	Message
13	10:20:41	CREATE PROCEDURE DepositMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected
14	10:20:41	CALL DepositMoney(10001, 5000)	1 row(s) affected

PROCEDURE TO WITHDRAW MONEY

The screenshot shows the MySQL Workbench interface with the '2520080053_Ruhitha' database selected. The 'SCHEMAS' panel on the left shows the database structure. The main editor displays the following SQL code:

```
59 IN p_Amount DECIMAL(12,2)
60 )
61 BEGIN
62
63 UPDATE Account
64 SET Balance = Balance + p_Amount
65 WHERE Account_No = p_Account_No;
66
67 END //
68 DELIMITER ;
69 CALL DepositMoney(10001, 5000);
70
71 DELIMITER //
72 CREATE PROCEDURE WithdrawMoney(
73 IN p_Account_No INT,
74 IN p_Amount DECIMAL(12,2)
75 )
76 BEGIN
77
78 UPDATE Account
79 SET Balance = Balance - p_Amount
80 WHERE Account_No = p_Account_No;
81
82 END //
83 DELIMITER ;
84 CALL WithdrawMoney(10001, 3000);
```

The 'Output' panel at the bottom shows the execution results:

#	Time	Action	Message
15	10:23:32	CREATE PROCEDURE WithdrawMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN ...	0 row(s) affected
16	10:23:32	CALL WithdrawMoney(10001, 3000)	1 row(s) affected

TRIGGERS

TRIGGER – PREVENT NEGATIVE BALANCE

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor window shows a SQL script with the following queries:

```
77 SET Balance = Balance - p_Amount
78 -- Execute the selected portion of the script or everything, if there is no selection
79 END //
80 DELIMITER ;
81 CALL WithdrawMoney(10001, 3000);
82
83
84 DELIMITER //
85 CREATE TRIGGER CheckBalance
86 BEFORE UPDATE ON Account
87 FOR EACH ROW
88 BEGIN
89     IF NEW.Balance < 0 THEN
90         SIGNAL SQLSTATE '45000'
91         SET MESSAGE_TEXT =
92         'Transaction failed: Insufficient balance';
93     END IF;
94 END //
95 DELIMITER ;
96 UPDATE Account
97 SET Balance = Balance - 60000
98 WHERE Account_No = 10001;
99
```

The 'Output' tab at the bottom shows the execution results:

#	Time	Action	Message
18	10:28:24	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Bal...	0 row(s) affected
19	10:28:24	UPDATE Account SET Balance = Balance - 60000 WHERE Account_No = 10001	Error Code: 1644. Transaction failed: Insufficient balance

TRIGGER – PREVENT NEGATIVE DEPOSIT

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor window shows a SQL script with the following queries:

```
94 END //
95 DELIMITER ;
96 UPDATE Account
97 SET Balance = Balance - 60000
98 WHERE Account_No = 10001;
99
100 DELIMITER //
101 CREATE TRIGGER CheckTransactionAmount
102 BEFORE INSERT ON Bank_Transaction
103 FOR EACH ROW
104 BEGIN
105     IF NEW.Amount <= 0 THEN
106         SIGNAL SQLSTATE '45000'
107         SET MESSAGE_TEXT =
108         'Transaction amount must be greater than zero';
109     END IF;
110 END //
111 DELIMITER ;
112 INSERT INTO Bank_Transaction
113 (Account_No, Transaction_Type, Amount)
114 VALUES
115 (10001, 'DEPOSIT', -5000);
116
```

The 'Output' tab at the bottom shows the execution results:

#	Time	Action	Message
20	10:29:42	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW B...	0 row(s) affected
21	10:29:42	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', -50...	Error Code: 1644. Transaction amount must be greater than zero

CREATE TRANSACTION AUDIT TABLE

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'CHEMAS' tree with the '2520080053_ruhitha' database selected. The right pane shows the 'Query 1' window with the following SQL script:

```
104 BEGIN
105 IF NEW.Amount <= 0 THEN
106     SIGNAL SQLSTATE '45000'
107     SET MESSAGE_TEXT =
108     'Transaction amount must be greater than zero';
109 END IF;
110 END //
111 DELIMITER ;
112 INSERT INTO Bank_Transaction
113 (Account_No, Transaction_Type, Amount)
114 VALUES
115 (10001, 'DEPOSIT', -5000);
116
117
118 CREATE TABLE Transaction_Audit (
119     Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
120     Transaction_ID INT,
121     Account_No INT,
122     Transaction_Type VARCHAR(20),
123     Amount DECIMAL(12,2),
124     Audit_Date DATETIME DEFAULT CURRENT_TIMESTAMP
125 );
126
```

The 'Output' pane at the bottom shows the execution results:

#	Time	Action	Message
21	10:29:42	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', -50...	Error Code: 1644, Transaction am
22	10:30:39	CREATE TABLE Transaction_Audit (Audit_ID INT PRIMARY KEY AUTO_INCREMENT, Transaction_ID...	0 row(s) affected

TRIGGER – TRANSACTION AUDIT

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'SCHEMAS' tree with the '2520080053_ruhitha' database selected. The right pane shows the 'Query 1' window with the following SQL script:

```
139 )
140 VALUES
141 (
142     NEW.Transaction_ID,
143     NEW.Account_No,
144     NEW.Transaction_Type,
145     NEW.Amount
146 );
147 END //
148 DELIMITER ;
149 INSERT INTO Bank_Transaction
150 (Account_No, Transaction_Type, Amount)
151 VALUES
```

The 'Result Grid' pane shows the data in the 'Transaction_Audit' table:

Audit_ID	Transaction_ID	Account_No	Transaction_Type	Amount	Audit_Date
1	23	10001	DEPOSIT	2500.00	2026-08-14 10:33:05

The 'Output' pane at the bottom shows the execution results:

#	Time	Action	Message
21	10:29:42	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', -50...	Error Code: 1644, Transaction amount must be greater than zero
22	10:30:39	CREATE TABLE Transaction_Audit (Audit_ID INT PRIMARY KEY AUTO_INCREMENT, Transaction_ID...	0 row(s) affected
23	10:33:05	CREATE TRIGGER TransactionAudit AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGIN IN...	0 row(s) affected
24	10:33:05	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', 25...	1 row(s) affected
25	10:33:05	SELECT * FROM Transaction_Audit LIMIT 0, 1000	1 row(s) returned

TRIGGER – AUTOMATICALLY UPDATE ACCOUNT BALANCE

The screenshot displays the MySQL Workbench interface. The top toolbar includes icons for File, Edit, View, Query, Database, Server, Tools, Scripting, and Help. The left sidebar shows the 'SCHEMAS' tree with a filter 'Filter objects'. The tree includes a database '2520080053_ruhitha' with sub-items: Tables, Views, Stored Procedures, Functions, bookflow_db, kludb, and sys. The main editor shows a SQL script with the following code:

```
166 SET Balance = Balance - NEW.Amount
167 WHERE Account_No = NEW.Account_No;
168 END IF;
169 END //
170 DELIMITER ;
171 INSERT INTO Bank_Transaction
172 (Account_No, Transaction_Type, Amount)
173 VALUES
174 (10001, 'DEPOSIT', 5000);
175 SELECT *
176 FROM Account
177 WHERE Account_No = 10001;
```

Below the editor is the 'Result Grid' showing the execution results of the queries. The grid has columns: Account_No, Customer_ID, Account_Type, Balance, and Branch. The data shown is:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	27000.00	Hyderabad

At the bottom, the 'Output' tab is selected, showing the 'Action Output' for the executed queries. The output table has columns: #, Time, Action, and Message. The messages are:

#	Time	Action	Message
24	10:33:05	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', 25...	1 row(s) affected
25	10:33:05	SELECT * FROM Transaction_Audit LIMIT 0, 1000	1 row(s) returned
26	10:35:24	CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
27	10:35:24	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', 50...	1 row(s) affected
28	10:35:24	SELECT * FROM Account WHERE Account_No = 10001 LIMIT 0, 1000	1 row(s) returned

TRIGGER – PREVENT WITHDRAWAL ABOVE BALANCE

2520080053_Ruhitha (252008005... x 2520080053_Ruhitha (25200... x Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator: Filter objects

SCHEMAS

- 2520080053_ruhitha
 - Tables
 - Views
 - Stored Procedures
 - Functions
 - bookflow_db
 - kludb
 - sys

Query 1 SQL File 1* SQL File 2* SQL File 3* x

```

182 BEFORE INSERT ON Bank_Transaction
183 FOR EACH ROW
184 BEGIN
185     DECLARE CurrentBalance DECIMAL(12,2);
186     SELECT Balance
187     INTO CurrentBalance
188     FROM Account
189     WHERE Account_No = NEW.Account_No;
190     IF NEW.Transaction_Type = 'WITHDRAW'
191     AND NEW.Amount > CurrentBalance THEN
192         SIGNAL SQLSTATE '45000'
193         SET MESSAGE_TEXT =
194         'Withdrawal failed: Insufficient balance';
195     END IF;
196 END //
197 DELIMITER ;
198 INSERT INTO Bank_Transaction
199 (Account_No, Transaction_Type, Amount)
200 VALUES
201 (10001, 'WITHDRAW', 1000000);

```

Administration Schemas Information

No object selected

Output

Action Output

#	Time	Action	Message
25	10:33:05	SELECT * FROM Transaction_Audit LIMIT 0, 1000	1 row(s) returned
26	10:35:24	CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH RO...	0 row(s) affected
27	10:35:24	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', 50...	1 row(s) affected
28	10:35:24	SELECT * FROM Account WHERE Account_No = 10001 LIMIT 0, 1000	1 row(s) returned
29	10:39:10	CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH RO...	Error Code: 1359. Trigger already exists

ADVANCED STORED PROCEDURES

PROCEDURE – ACCOUNT TRANSFER

2520080053_Ruhitha (252008005... x 2520080053_Ruhitha (25200... x Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator: Filter objects

SCHEMAS

- 2520080053_ruhitha
 - Tables
 - Views
 - Stored Procedures
 - Functions
 - bookflow_db
 - kludb
 - sys

Query 1 SQL File 1* SQL File 2* SQL File 3* x

```

221 SET Balance = Balance - TransferAmount
222 WHERE Account_No = SenderAccount;
223 UPDATE Account
224 SET Balance = Balance + TransferAmount
225 WHERE Account_No = ReceiverAccount;
226 END IF;
227 END //
228 DELIMITER ;
229 CALL TransferMoney(10001, 10002, 5000);
230 SELECT *
231 FROM Account
232 WHERE Account_No IN (10001,10002);

```

Administration Schemas Information

No object selected

Result Grid

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	22000.00	Hyderabad
10002	102	Savings	117000.00	Vijayawada
NULL	NULL	NULL	NULL	NULL

Account 10 x

Output

Action Output

#	Time	Action	Message
28	10:35:24	SELECT * FROM Account WHERE Account_No = 10001 LIMIT 0, 1000	1 row(s) returned
29	10:39:10	CREATE TRIGGER UpdateBalanceAfterTransaction AFTER INSERT ON Bank_Transaction FOR EACH RO...	Error Code: 1359. Trigger already exists
30	11:00:53	CREATE PROCEDURE TransferMoney(IN SenderAccount INT, IN ReceiverAccount INT, IN Transfer...	0 row(s) affected
31	11:00:53	CALL TransferMoney(10001, 10002, 5000)	1 row(s) affected
32	11:00:54	SELECT * FROM Account WHERE Account_No IN (10001,10002) LIMIT 0, 1000	2 row(s) returned

PROCEDURE – CUSTOMER LOAN DETAILS

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor window shows a SQL script for a procedure named 'GetCustomerLoans'. The script is as follows:

```
240 C.Customer_Name,  
241 L.Loan_ID,  
242 L.Loan_Type,  
243 L.Loan_Amount,  
244 L.Interest_Rate  
245 FROM Customer C  
246 JOIN Loan L  
247 ON C.Customer_ID = L.Customer_ID  
248 WHERE C.Customer_ID = p_Customer_ID;  
249 END //  
250 DELIMITER ;  
251 CALL GetCustomerLoans(101);
```

The 'Result Grid' at the bottom displays the output of the procedure call, showing a single row of loan details for Customer ID 101:

Customer_Name	Loan_ID	Loan_Type	Loan_Amount	Interest_Rate
Ravi Kumar	501	Home Loan	5000000.00	7.50

The 'Output' pane at the bottom shows the execution log, including the creation of the procedure and the successful execution of the call.

PROCEDURE – HIGH BALANCE ACCOUNTS

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor window shows a SQL script for a procedure named 'HighBalanceAccounts'. The script is as follows:

```
254 CREATE PROCEDURE HighBalanceAccounts(  
255 IN MinimumBalance DECIMAL(12,2)  
256 )  
257 BEGIN  
258 SELECT *  
259 FROM Account  
260 WHERE Balance >= MinimumBalance  
261 ORDER BY Balance DESC;  
262 END //  
263 DELIMITER ;  
264 CALL HighBalanceAccounts(50000);  
265
```

The 'Result Grid' at the bottom displays the output of the procedure call, showing a list of accounts with balances greater than or equal to 50,000:

Account_No	Customer_ID	Account_Type	Balance	Branch
10007	107	Current	150000.00	Mumbai
10003	103	Current	130000.00	Bangalore
10002	102	Savings	117000.00	Vijayawada
10010	110	Current	110000.00	Vijayawada
10005	105	Current	90000.00	Hyderabad

The 'Output' pane at the bottom shows the execution log, including the creation of the procedure and the successful execution of the call.

PROCEDURE WITH IN, OUT PARAMETERS

The screenshot displays the MySQL Workbench interface. The left sidebar shows the 'SCHEMAS' tree with the database '2520080053_ruhitha' selected. The main editor shows a SQL script for a stored procedure named 'GetBalance'.

```
269 OUT p_Balance DECIMAL(12,2)
270 )
271 BEGIN
272     SELECT Balance
273     INTO p_Balance
274     FROM Account
275     WHERE Account_No = p_Account_No;
276 END //
277 DELIMITER ;
278 CALL GetBalance(10001, @CurrentBalance);
279 SELECT @CurrentBalance;
280
```

Below the script, the 'Result Grid' shows the output of the procedure call:

	@CurrentBalance
▶	22000.00

The bottom section shows the 'Output' tab with 'Action Output' selected. It displays a log of database actions:

#	Time	Action	Message
✓ 35	11:06:08	CREATE PROCEDURE HighBalanceAccounts(IN MinimumBalance DECIMAL(12,2)) BEGIN SELECT * ...	0 row(s) affected
✓ 36	11:06:08	CALL HighBalanceAccounts(50000)	7 row(s) returned
✓ 37	11:08:17	CREATE PROCEDURE GetBalance(IN p_Account_No INT, OUT p_Balance DECIMAL(12,2)) BEGIN ...	0 row(s) affected
✓ 38	11:08:17	CALL GetBalance(10001, @CurrentBalance)	1 row(s) affected
✓ 39	11:08:17	SELECT @CurrentBalance LIMIT 0, 1000	1 row(s) returned